

Week-12

Why so many algorithms for classification? Common framework?

Dataset = $\{(x_1, y_1) \dots (x_n, y_n)\}$ $x_i \in \mathbb{R}^d$ $y_i \in \{+1, -1\}$

Performance measure $\sum_{i=1}^n \mathbb{1}(h(x_i) \neq y_i)$ ie # mistakes

Restrict h to a certain class of functions & search in that space

$$\min_{\substack{w \in \mathbb{R}^d \\ b \in \mathbb{R}}} \sum_{i=1}^n \mathbb{1}(\text{sign}(w^T x_i + b) \neq y_i)$$

→ Can add an extra constant feature
and the weight of that feature = bias

NP-HARD problem

(if dataset is not linearly separable)

⇒ Make assumptions about the data

Eg: Perceptron / Generative models

Loss Function View : How much loss does a single (x, y)
suffer wrt an underlying h ?

(x, y) , $h: \mathbb{R}^d \rightarrow \{\pm 1\}$

Loss is either 0 or 1

$$\mathbb{1}(h(x) \neq y) = \mathbb{1}(h(x) \cdot y < 0) \quad \text{or} \quad \mathbb{1}(\underbrace{(w^T x)}_{\text{sign}(w^T x)} \cdot y < 0)$$

$\text{sign}(w^T x)$ $w^T x \geq 0$: $y = +1$: +ve
 $w^T x < 0$: $y = -1$: +ve

→ if both have same sign
product is positive



$$h(x) = \text{sign}(g(x))$$

$\sum_{i=1}^n \mathbb{1}(g(x_i) \cdot y_i < 0) \Rightarrow$ This is a sum of a bunch of these individual loss functions

This loss function is discontinuous + non-convex

If it was convex, then $\sum(\text{convex fns}) = \text{convex fn}$ which can be minimized

Different algorithms deal with this in different ways

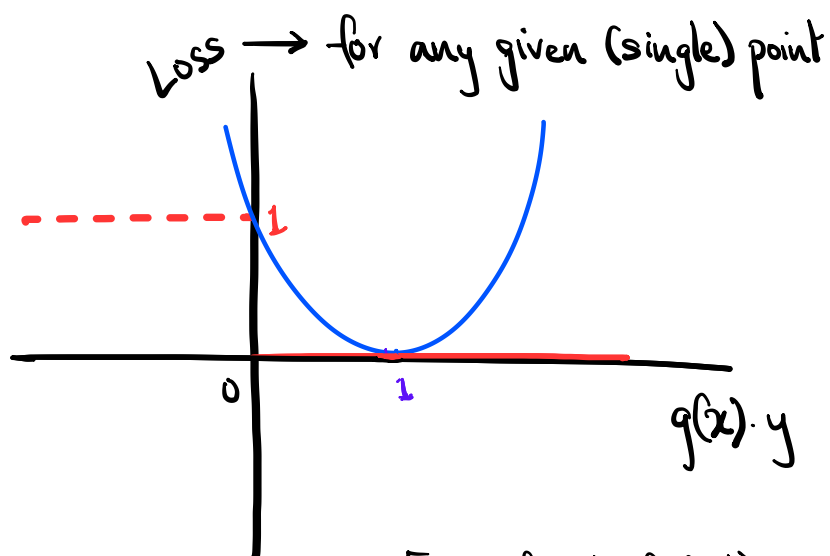
Algorithm 1 : Using Regression for classification

$$\text{Loss}(g, (x, y)) = (g(x) - y)^2 \rightarrow \{\pm 1\} \quad h(x) = \text{sign}[g(x)]$$

$$\begin{aligned} &= (g(x) \cdot y - 1)^2 \\ &= (g(x)y)^2 + 1 - 2g(x) \cdot y \\ &= g(x)^2 + 1 - 2g(x) \cdot y \end{aligned}$$

$$\begin{aligned} &g(x)^2 + y^2 - 2g(x) \cdot y \\ &= g(x)^2 + 1 - 2g(x) \cdot y \end{aligned}$$

$\therefore (g(x)y - 1)^2$ is another way to write $(g(x) - y)^2$



$(g(x) \cdot y - 1)^2$: Squared loss

Zero-One loss

Even if $\text{sign}(g(x)) = y$, (ie) correct prediction,
the loss is large if $|g(x)|$ is large

$\therefore$ This is a bad way to approximate the Zero-One loss

SVM and Hinge loss

$$\min_{w, \varepsilon} \quad \frac{1}{2} \|w\|^2 + c \cdot \sum_{i=1}^n \varepsilon_i \quad \text{such that} \quad \begin{aligned} (w^T x_i) y_i + \varepsilon_i &\geq 1 \\ \varepsilon_i &\geq 0 \end{aligned}$$

$$\min_{w, \varepsilon} \quad \frac{1}{2} \|w\|^2 + c \cdot \sum_{i=1}^n \varepsilon_i \quad \text{such that} \quad \left. \begin{aligned} \varepsilon_i &\geq 1 - (w^T x_i) y_i \\ \varepsilon_i &\geq 0 \end{aligned} \right\}$$

$$\varepsilon_i \geq \max(0, 1 - (w^T x_i) y_i)$$

$$\equiv \min_w \quad \underbrace{\frac{1}{2} \|w\|^2}_{\text{Model dependent (regularizer)}} + c \cdot \sum_{i=1}^n \underbrace{\max(0, 1 - (w^T x_i) y_i)}_{\text{Data dependent (loss)}} \rightarrow g(x_i)$$

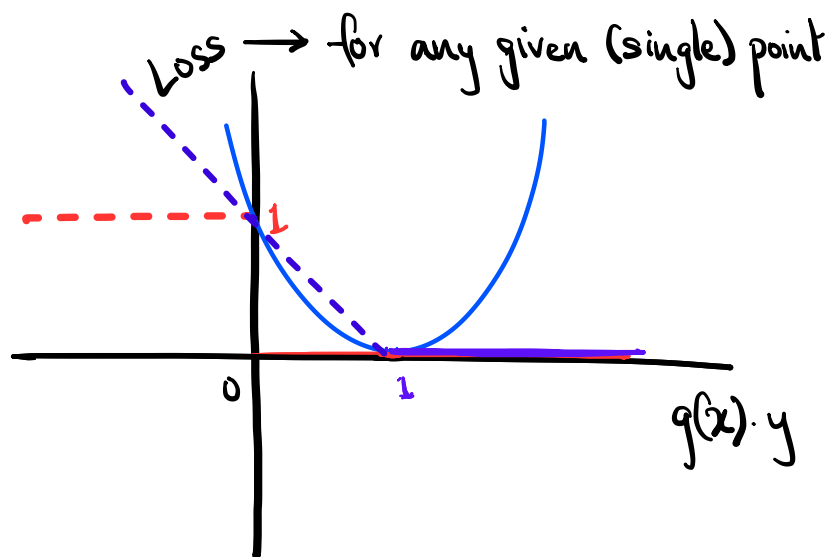
Model dependent
(regularizer)

Data dependent
(loss)

$$(w^T x_i) y_i > 1 : 0$$

$$(w^T x_i) y_i = 1 : 0$$

$$(w^T x_i) y_i < 1 : 1 - (w^T x_i) y_i \rightarrow \text{some positive number}$$



Zero-One loss

Squared loss

$\max(0, 1 - g(x) \cdot y)$
 $\equiv$ Hinge-loss

Hinge-loss is the best convex approximation
of the zero-one loss

$$\min_{\omega} \underbrace{\frac{1}{2} \|\omega\|^2}_{L_2 \text{ regularization}} + c \cdot \sum_{i=1}^n \underbrace{\max(0, 1 - (\omega^T x_i) y_i)}_{\text{Hinge-loss}} \rightarrow g(x_i)$$

$\therefore$ SVM is actually minimizing this Hinge-loss

Logistic Regression & Logistic Loss

$$\max_{\omega} \prod_{i=1}^n (\sigma(\omega^T x_i))^{z_i} (1 - \sigma(\omega^T x_i))^{1-z_i}$$

$$\max_{\omega} \sum_{i=1}^n z_i \log(\sigma(\omega^T x_i)) + (1 - z_i) \log(1 - \sigma(\omega^T x_i))$$

$$\equiv \min_{\omega} \sum_{i=1}^n -z_i \log(\sigma(\omega^T x_i)) - (1 - z_i) \log(1 - \sigma(\omega^T x_i))$$

$$\sigma(a) = \frac{1}{1 + e^{-a}}$$

$$z_i \in \{0, 1\}$$

$$z_i = 0 \text{ if } y_i = -1$$

$$z_i = 1 \text{ if } y_i = +1$$

Loss for a single point when $z_i = 1$: (ie $y_i = +1$)

$$-\log(\sigma(w^T x_i)) = -\log\left(\frac{1}{1+e^{-w^T x_i}}\right) = \log(1+e^{-w^T x_i})$$

Equivalently: $\log(1+e^{(w^T x_i) y_i})$

Loss for a single point when $z_i = 0$: (ie $y_i = -1$)

$$-\log(1-\sigma(w^T x_i)) = -\log\left(1-\frac{1}{1+e^{-w^T x_i}}\right) = -\log\left(\frac{e^{-w^T x_i}}{1+e^{-w^T x_i}}\right)$$

$$= -\log\left(\frac{1}{e^{w^T x_i} + 1}\right)$$

[dividing by $e^{-w^T x_i}$ for both N/D]

$$= \log(1+e^{w^T x_i}) = \log(1+e^{(w^T x_i) y_i})$$

Same in both cases !

$\therefore$ Logistic Regression can be thought of as minimizing this particular loss function

$$\min_w \sum_{i=1}^n \log(1+e^{-w^T x_i y_i})$$



$\log(1+e^{-g(x) \cdot y})$
ie Logistic loss

$\therefore$ Different algorithms use different convex "surrogates" to minimize instead of the zero-one loss and depending on how good this surrogate is in representing the original zero-one loss, we get different performances

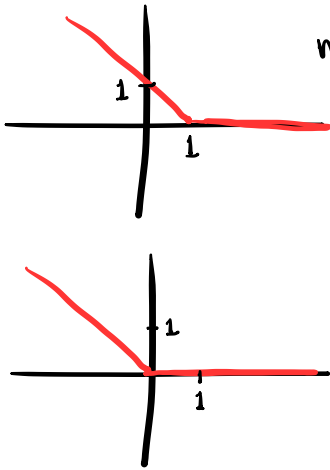
In each of these cases, we can add a regularization term

Perceptron

$$w_{t+1} = w_t + x_t \cdot y_t \quad (\text{looks like gradient update rule})$$

Hinge loss : $\ell_{\text{hinge}}(w, (x, y)) = \max(0, 1 - (w^T x)y)$

$$\text{mod. } \ell_{\text{hinge}}(w, (x, y)) = \max(0, -(w^T x)y)$$



$$\nabla_w \ell_{\text{hinge}}^{\text{(mod)}} = \begin{cases} -x \cdot y & , \text{ if } (w^T x)y < 0 \\ 0 & , \text{ if } (w^T x)y > 0 \\ [-1, 0] \cdot xy & , \text{ if } (w^T x)y = 0 \end{cases}$$

$\text{ie: Any } c \cdot xy \text{ where } c \in [-1, 0] \text{ is a valid sub-gradient to choose}$
So we can simply pick $c = -1$

General Gradient Update

$$w_{t+1} = w_t - \eta_t \nabla_w \text{loss}(w_t)$$

for perceptron : $w_{t+1} = w_t - (-x_t y_t)$

$$\text{ie } \eta_t = 1$$

$\rightarrow$ (Stochastic GD)

$\therefore$ Perceptron takes 1 point at a time, does a gradient descent / sub-GD using a modified hinge loss + constant step-size = 1

Boosting

$$\text{Loss}(h, (x, y)) = \underline{e^{-y \cdot h(x)}} \rightarrow \text{Exponential loss}$$

Trying to add a new classifier that minimizes this exponential loss in every round

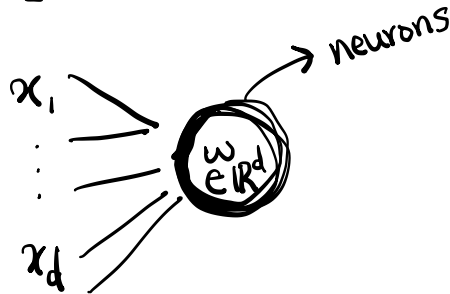
Neural Networks

$$x \in \mathbb{R}^d$$

$$\downarrow$$

$$[x_1 \dots x_d]$$

$$\text{sign}(w^T x)$$



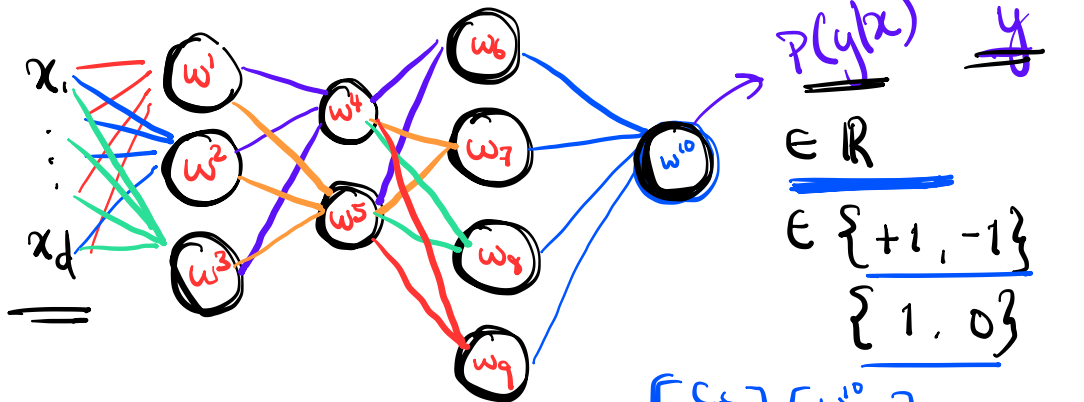
$$\begin{bmatrix} x_1 \\ \vdots \\ x_d \end{bmatrix} \begin{bmatrix} w_1 \\ \vdots \\ w_d \end{bmatrix} = c \in \mathbb{R}$$

$$\xrightarrow{w}$$

$$\text{sign}(c) \geq 0 : +1$$

$$-1$$

This is what Perceptron does



$$p(y|x)$$

$$\in \mathbb{R}$$

$$\in \{+1, -1\}$$

$$\{1, 0\}$$

$$\begin{bmatrix} c_1 \\ c_2 \\ c_3 \\ c_4 \\ c_5 \end{bmatrix} \begin{bmatrix} w_1^0 \\ w_2^0 \\ w_3^0 \\ w_4^0 \\ w_5^0 \end{bmatrix} = \mathbb{R}$$

$$x^T \cdot w^1 = c_1$$

$$x \in \mathbb{R}^d$$

$$x^T \cdot w^2 = c_2$$

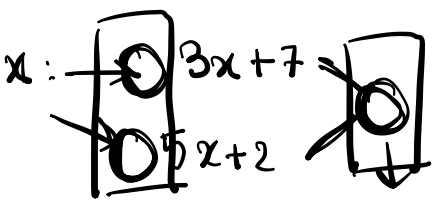
$$x^T \cdot w^3 = c_3$$

$$\begin{bmatrix} c_1 \\ c_2 \\ c_3 \end{bmatrix} \cdot \begin{bmatrix} w_1^4 \\ w_2^4 \\ w_3^4 \end{bmatrix} = c_4$$

$$\begin{bmatrix} c_1 \\ c_2 \\ c_3 \end{bmatrix} \cdot \begin{bmatrix} w_1^5 \\ w_2^5 \\ w_3^5 \end{bmatrix} = c_5$$

$$\begin{bmatrix} c_4 \\ c_5 \end{bmatrix} \cdot \begin{bmatrix} w_6^1 \\ w_7^1 \end{bmatrix}$$

$$\begin{bmatrix} c_4 \\ c_5 \end{bmatrix} \cdot \begin{bmatrix} w_8^1 \\ w_9^1 \end{bmatrix}$$



$$(2x_1 + 3x_2 + 5)$$

$$(w^T x + b)$$

$$w^T x \rightarrow \underline{w^T x + b} \checkmark$$

$$= 2(3x+7) + 3(5x+2) + 5 = 6x + 14 + 15x + 6 + 5$$

$$= 21x + 25$$

$$x \rightarrow \bigcirc : 21x + 25$$

Activation Functions

$$\begin{bmatrix} x_1 \\ \vdots \\ x_d \end{bmatrix} \begin{bmatrix} w_1 \\ \vdots \\ 1 \end{bmatrix} + b = c_1$$

$x \in \mathbb{R}^d \quad w_i \in \mathbb{R}^d$

$\downarrow$
 $g(c_1)$

$$g(x) = \text{activation}$$

$$x \rightarrow g(x)$$

$$\mathbb{R} \rightarrow \mathbb{R}$$

$$\underline{x \rightarrow \max(0, x)} : g(c) \rightarrow \max(0, c)$$

$$\text{Sigmoid} : \frac{1}{1+e^{-x}} : g(c) \rightarrow \frac{1}{1+e^{-c}} \in \underline{(0, 1)}$$

$$\text{Softmax} : \rightarrow (0, 1)$$

$$\tanh : \rightarrow (-1, 1)$$

$$w^T x + b = c_1$$

$$\underline{a, b, c, d}$$

↓ ↓ ↓ ↓

$$\begin{bmatrix} g(c_1) \\ g(c_2) \\ g(c_3) \end{bmatrix} \cdot \begin{bmatrix} w_1^4 \\ w_2^4 \\ w_3^4 \end{bmatrix} = c_4$$

$$\left(\frac{e^a}{e^a + e^b + e^c + e^d} \right) \frac{e^b}{e^a + e^b + e^c + e^d} \left(\frac{e^c}{e^a + e^b + e^c + e^d} \right) \left(\frac{e^d}{e^a + e^b + e^c + e^d} \right) \begin{bmatrix} g(c_4) \\ g(c_5) \end{bmatrix} \begin{bmatrix} w_1^6 \\ w_2^6 \end{bmatrix}$$

$\in (0, 1) \quad \in (0, 1) \quad \in (0, 1) \quad \in (0, 1)$

$$\left. \begin{array}{l} x, y=1 \\ \boxed{P(y=1|x) = 0.75} \\ \boxed{P(y=0|x) = 0.25} \end{array} \right| - (y \cdot \log p + (1-y) \log(1-p)) \dots$$

$= -\log(\text{prediction for correct output})$

$$= -\log(0.75)$$

$y=0$:

$$P(y=1|x) = 0.75$$

$$P(y=0|x) = 0.25$$

$$= -\log(0.25)$$

$$P(y=A|x)$$

$$P(y=B|x)$$

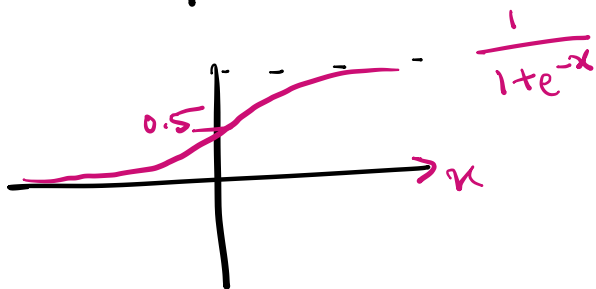
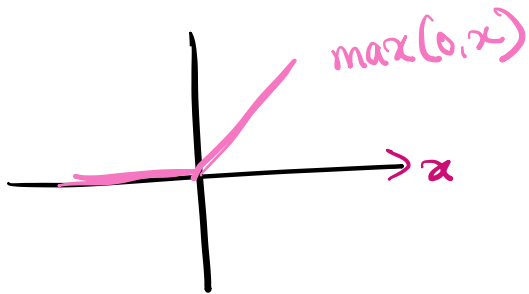
$$P(y=C|x)$$

$$P(y=D|x)$$



$$y=C \Rightarrow -\log(P(y=C|x))$$

$$y=D \Rightarrow -\log(P(y=D|x))$$



$$x \rightarrow (\underbrace{NN(x)}_{\hat{y}} - y)^2$$

Squared loss

$$\sum_{i=1}^n (\underbrace{NN(x)}_{\hat{y}} - y)^2$$

Maybe the real global optimum
was all the elegant algorithms
we learned along the way!